HASH TABLE:

Based on the provided lecture slides and your exam suggestion, here are the topics and necessary information for **Hashing**. This information is structured to help you with definitions, efficiency, pseudo-code, and simulations.

1. Definition and Core Concepts

- **Hash Table**: A data structure that stores key-value pairs in an array-like format for fast retrieval. It uses a **Hash Function** to compute an index for storing or finding a value.
- **Hashing**: A technique used to implement hash tables.
- **Collision**: A situation where two different keys hash to the same index.
- **Load Factor (λ)**: The ratio of the number of items (N) to the $TableSize$ ($\lambda = N / TableSize$). It represents the average length of the lists in separate chaining.

2. Analysis and Complexity

- **Average Case Complexity**: $O(1)$ for insertions, deletions, and searching.
- **Inefficient Operations**: Hashing is not efficient for finding minimum/maximum values or printing data in sorted order (it is not suitable when ordering is required).
- **Worst Case**: Search time can become $O(n)$ if collisions are not managed well (e.g., long chains in separate chaining).

3. Collision Resolution Strategies

There are two main categories: **Open Hashing (Separate Chaining)** and **Closed Hashing (Open Addressing)**.

A. Separate Chaining

- **Idea**: Maintain a linked list for all items that hash to the same value.
- **Insertion**: Items are inserted at the **front** of the list to maintain $O(1)$ efficiency.
- **Pros**: Simple to implement; the table never "fills up" (λ can be > 1).
- **Cons**: Uses extra space for pointers; search time increases as chains get longer.
- **Cost of Search**:
  - Unsuccessful: $O(1 + \lambda)$.
  - Successful: $O(1 + \lambda/2)$.

B. Open Addressing

All items are stored within the hash table itself; no external lists are used. If a collision occurs, alternative cells are tried using $h_i(x) = (hash(x) + f(i)) \mod TableSize$.

1. **Linear Probing**: $f(i) = i$. It searches cells sequentially.

    - **Problem**: **Primary Clustering**—blocks of occupied cells form, making collisions more frequent and costly.
2. **Quadratic Probing**: $f(i) = i^2$. It examines cells at distances of 1, 4, 9, etc., from the original probe.
    - **Problem**: **Secondary Clustering**—keys hashing to the same location follow the same probe sequence.
    - **Requirement**: To guarantee an empty space is found, $TableSize$ should be prime and $\lambda < 0.5$.
3. **Double Hashing**: $f(i) = i \times hash_2(x)$. A second hash function determines the probe distance.
    - **Requirement**: $hash_2(x)$ must never evaluate to zero. A common choice is $hash_2(x) = R - (x \mod R)$ where R is a prime smaller than $TableSize$.

4. Pseudo-code and Algorithmic Steps

Based on the operations described for **Separate Chaining** :

- **Initialize**: Set all table entries to `NULL`.
- **Search**:
    1. Apply `Hash(key)` to find the index.
    2. Perform a sequential search on the linked list at that index.
- **Insertion**:
    1. Apply `Hash(key)` to find the index.
    2. Insert the new item as the first node in the corresponding linked list.
- **Deletion**:
    1. Apply `Hash(key)` to find the index.
    2. Locate and delete the item from the linked list.

5. Simulation Examples (for your exam prep)

- **Linear Probing Simulation**: To insert 89 and 59 into a table of size 10:
    - `89 % 10 = 9`. Index 9 is empty; insert 89.
    - `59 % 10 = 9`. Collision! Try $i=1$: $(59 + 1) \mod 10 = 0$. Index 0 is empty; insert 59.
- **Quadratic Probing Simulation**: To insert 58 into a table of size 10 (with 89 at index 9 and 18 at index 8):
    - `58 % 10 = 8`. Collision!
    - Try $i=1$: $(58 + 1^2) \mod 10 = 9$. Collision!
    - Try $i=2$: $(58 + 2^2) \mod 10 = 2$. Index 2 is empty; insert 58 .

6. Design and Theoretical Reasoning

- **Good Hash Function Requirements**:

    1. Simple to compute ($O(1)$).
    2. Distribute keys evenly to minimize collisions.
    3. Should not waste space.
- **Re-hashing**: When the load factor becomes too high (e.g., $\lambda > 0.5$ in quadratic probing), the table size is doubled to a new prime number, and all items are re-inserted.

Binary heap:

Based on the provided lecture slides and your semester final suggestion, here is the necessary information for the topics of **Hashing** and **Binary Heap**.

Topic 1: Hashing

1. Definition & Core Concepts

- **Hash Table**: A data structure that stores key-value pairs in an array-like format for fast retrieval.
- **Hash Function**: Used to compute an index from a key to store or find values.
- **Collision**: Occurs when two different keys map to the same index in the hash table.
- **Load Factor (λ)**: The ratio of total items (N) to the $TableSize$. For separate chaining, λ is the average length of the lists.

2. Complexity Analysis

- **Average Case**: $O(1)$ for Insertion, Deletion, and Searching.
- **Inefficient Operations**: Hashing is not suitable for finding minimum/maximum values or printing data in sorted order.
- **Worst Case**: For Separate Chaining, if a chain becomes long, search time can become $O(N)$.

3. Collision Resolution Strategies

- **Separate Chaining (Open Hashing)**:
    - **Method**: Each array cell is a pointer to the head of a linked list containing items that hashed to that index.
    - **Insertion**: Items are inserted at the **front** of the list for $O(1)$ efficiency.
    - **Search Cost**: $O(1 + \lambda)$ for unsuccessful searches and $O(1 + \lambda/2)$ for successful searches.
- **Open Addressing (Closed Hashing)**:
    - All items are stored within the table itself; alternative cells are tried using $h_i(x) = (hash(x) + f(i)) \mod TableSize$.

- **Linear Probing**: $f(i) = i$. Probes sequentially. **Issue**: Primary Clustering (blocks of occupied cells).
- **Quadratic Probing**: $f(i) = i^2$. Probes at $1, 4, 9...$ cells away. **Advantage**: Eliminates primary clustering.
- **Double Hashing**: $f(i) = i \times hash_2(x)$. Uses a second hash function to determine probe distance.

4. Pseudo-code & Operations

- **Initialize**: Set all table entries to `NULL`.
- **Search/Insert/Delete**:
  1. Apply the Hash function to find the index.
  2. Traverse the list (Chaining) or follow the probe sequence (Open Addressing) to find or place the item.

Topic 2: Binary Heap

1. Definition & Properties

- **Binary Heap**: A complete binary tree used to balance $O(\log N)$ insertion and $O(\log N)$ extraction.
- **Heap Property (Max-Heap)**: For every node i, the value of the parent is greater than or equal to the children. The largest element is always at the root.
- **Shape Property**: A Complete Binary Tree where all levels are fully filled except possibly the last, which is filled from left to right.

2. Analysis & Complexity

- **Priority Queue Implementation**:
  - **Insert**: $O(\log N)$.
  - **Extract Max/Min**: $O(\log N)$.
  - **Peek Max/Min**: $O(1)$.
- **Naive Comparison**: Unlike unsorted arrays (slow search) or sorted arrays (slow insertion), heaps balance both.

3. Operations & Simulation Steps

- **Percolate Up (Insertion)**:
  1. Place the new element at the first available leaf position (end of the array).
  2. Swap with its parent until the heap property is restored.
- **Percolate Down (Extract Max)**:
  1. Replace the root with the last leaf element.
  2. Decrease heap size.

    3. Swap the root with its larger child repeatedly until the heap property is restored.
- **Heap Sort**:
    1. Build a Max-Heap from an unordered array ($O(N)$).
    2. Repeatedly swap the root with the last element and call `max_heapify` on the root ($O(N \log N)$).

Summary Table for Scenario Choice

Data Structure	Best Use Case (Scenario)	Primary Advantage
Hashing	Fast lookup, insertion, and deletion by key.	$O(1)$ average time.
Binary Heap	Implementing Priority Queues; finding Max/Min repeatedly.	Balanced $O(\log N)$ for both add and extract.
Trie	String searching, prefix matching.	efficient for prefix queries.
AVL Tree	Maintaining sorted data with guaranteed $O(\log N)$ worst-case.	Strictly balanced.

AVL tree:

Based on the provided PDF and your semester final suggestion, here is the necessary information for **Set 2: AVL Trees (25 Marks)**. This covers definitions, complexity, rotation logic, and simulation steps.

Topic: AVL Tree (Self-Balancing Binary Search Tree)

1. Definition & Theoretical Reasoning

- **AVL Tree:** A self-balancing Binary Search Tree (BST) where the heights of the two child subtrees of any node differ by at most one.
- **Balance Factor (BF):** For any node, $BF = \text{Height(Left Subtree)} - \text{Height(Right Subtree)}$.
- **Property:** In an AVL tree, the Balance Factor of every node must be **-1, 0, or 1**.
- **Why AVL?** In a standard BST, if nodes are inserted in sorted order, the tree becomes a "skewed" line with $O(N)$ height. AVL trees ensure the height remains $O(\log N)$, guaranteeing efficient operations.

2. Complexity Analysis

Operation	Average Case	Worst Case
Search	$O(\log N)$	$O(\log N)$
Insertion	$O(\log N)$	$O(\log N)$
Deletion	$O(\log N)$	$O(\log N)$

- **Height:** The height of an AVL tree with n nodes is strictly limited to $1.44 \log n$.

3. Rotations (Operations to Restore Balance)

When an insertion or deletion causes a node to have a BF of $+2$ or -2, rotations are performed. There are four types:

A. Single Rotations (LL and RR):

- **LL Rotation (Left-Left):** Applied when a node is inserted into the left subtree of the left child. Rotate the unbalanced node to the **right**.
- **RR Rotation (Right-Right):** Applied when a node is inserted into the right subtree of the right child. Rotate the unbalanced node to the **left**.

B. Double Rotations (LR and RL):

- **LR Rotation (Left-Right):** Applied when a node is inserted into the right subtree of the left child.
    - *Step 1:* Left rotate the left child.
    - *Step 2:* Right rotate the unbalanced node.
- **RL Rotation (Right-Left):** Applied when a node is inserted into the left subtree of the right child.
    - *Step 1:* Right rotate the right child.
    - *Step 2:* Left rotate the unbalanced node.

4. Simulation Steps: Insertion

1. **Standard BST Insert:** Insert the node following BST rules (smaller to the left, larger to the right).
2. **Update Heights:** Traverse back up from the inserted node to the root, updating the height of each ancestor.
3. **Check Balance Factor:** At each ancestor, calculate BF.
4. **Rebalance:** If BF is out of range $(-2 \text{ or } +2)$, identify the rotation type (LL, RR, LR, RL) and perform it.

5. Simulation Steps: Deletion

1. **Standard BST Delete:** * If leaf: Remove it.
    - If one child: Replace node with child.
    - If two children: Replace with **In-order Successor** (smallest in right subtree) or **In-order Predecessor** (largest in left subtree).
2. **Update & Rebalance:** Similar to insertion, traverse up to the root and perform necessary rotations ($R0, R1, R-1$ variations as shown in the lecture slides).

6. Scenario Choice: When to use AVL?

- **Scenario:** You need a data structure for a system that requires frequent **lookups/searches** and the data must remain **sorted**.

- **Reasoning:** Use AVL instead of a standard BST because it prevents the tree from becoming skewed, ensuring that the search time is always $O(\log N)$ even in the worst case.
- **Efficiency:** Better worst-case search time than a standard BST and more strictly balanced than a Red-Black tree (though Red-Black is often faster for frequent insertions).

Preparation Tip for your Exam:

Based on the "Question Pattern" you provided:

- **Practice Drawing:** Be ready to simulate the insertion of a sequence of numbers (e.g., 10, 20, 30...) and show the tree after each rotation.
- **Pseudo-code:** Focus on the `rightRotate(y)` and `leftRotate(x)` functions, as they are the building blocks for the rebalancing logic.

Range queries and prefix sum:

Based on the provided lecture slides and your exam suggestion, here is the essential information for **Set 1: Range Queries**. This covers Prefix Sum and Sparse Table.

Topic 1: Prefix Sum

1. Definition & Theoretical Reasoning

- **Definition:** A technique to efficiently compute the sum of elements in a subarray.
- **Formula:** `pref[i]` represents the sum of the first `i` elements.
  - `pref[0] = 0`
  - `pref[i] = pref[i-1] + a[i-1]` (where `a` is 0-indexed)
- **Range Sum Query:** To find the sum of elements in range `[L, R]`:
  - **Sum(L, R) = pref[R+1] - pref[L]**

2. Complexity Analysis

- **Preprocessing Time:** $O(N)$ (one pass to build the prefix array).
- **Query Time:** $O(1)$ (constant time subtraction).
- **Space Complexity:** $O(N)$ to store the prefix array.

3. Pseudo-code

```
C++
```

```
// Preprocessing
void buildPrefixSum(int a[], int n, int pref[]) {
    pref[0] = 0;
    for (int i = 1; i <= n; i++) {
        pref[i] = pref[i-1] + a[i-1];
    }
}

// Query
int query(int L, int R, int pref[]) {
    return pref[R+1] - pref[L];
}
```

Topic 2: Sparse Table

1. Definition & Theoretical Reasoning

- **Definition:** A data structure used for **Range Minimum/Maximum Queries (RMQ)** on a static array. It relies on the idea that any integer can be represented as a sum of powers of 2.
- **Static Requirement:** This is only efficient if the array does not change (no updates).
- **Overlap Property:** For operations like `min`, `max`, and `gcd`, overlapping ranges do not affect the result. This allows $O(1)$ queries.

2. Analysis & Complexity

- **Preprocessing Time:** $O(N \log N)$.
- **Query Time:** $O(1)$ for idempotent functions (min, max, gcd).
- **Space Complexity:** $O(N \log N)$ to store the 2D table.

3. Pseudo-code & Operations

Preprocessing:

The table `st[i][j]` stores the answer for the range starting at index `i` with length 2^j.

C++

```
for (int i = 0; i < n; i++)
    st[i][0] = a[i];

for (int j = 1; j <= logN; j++) {
    for (int i = 0; i + (1 << j) <= n; i++) {
        st[i][j] = min(st[i][j-1], st[i + (1 << (j-1))][j-1]);
    }
}
```

Querying:

To find the minimum in range [L, R], find the largest power of 2 (2^j) that fits in the range.

```C++
int query(int L, int R) {
    int j = log2[R - L + 1];
    return min(st[L][j], st[R - (1 << j) + 1][j]);
}
```

---

4. Scenario Choice & Application

- **When to choose Prefix Sum?** Use it when the scenario asks for **Range Sums** and you need the fastest possible query time ($O(1)$) with minimal memory overhead.
- **When to choose Sparse Table?** Use it when the scenario asks for **Range Minimum/Maximum** on a large array that is **static** (no updates). It is more efficient for queries than a Segment Tree but cannot handle updates easily.
- **Simulation Tip:** For Sparse Table questions, you may be asked to fill a 2D table. Remember that st[i][1] is the minimum of two elements, st[i][2] is the minimum of four, and so on.

---

*(Note: Segment Tree is listed in your suggestion but was not the primary focus of the specific slides provided in this PDF. It typically handles range queries with **updates** in $O(\log N)$ time.)*

Disjoint sets:

Based on the provided lecture slides and your exam suggestion, here is the essential information for **DSU (Disjoint Set Union)**, which falls under the two sets of questions in category 3.

Topic: Disjoint Set Union (DSU) / Union-Find

1. Definition & Theoretical Reasoning

- **Definition:** A data structure that keeps track of a set of elements partitioned into a number of disjoint (non-overlapping) subsets.

- **Purpose:** Used in scenarios requiring dynamic grouping and connectivity checks (e.g., Kruskal's algorithm for Minimum Spanning Trees, cycle detection in graphs, or social network connectivity).
- **Core Operations:**
  - **MakeSet(x):** Creates a new set containing only the element x.
  - **Find(x):** Returns the representative (or "root") of the set containing x. This is used to determine if two elements belong to the same set.
  - **Union(x, y):** Merges the two sets containing x and y into a single set.

2. Complexity Analysis

- **Naive Implementation:** $O(N)$ for Find and Union in the worst case (if the structure becomes a skewed tree/linked list).
- **Optimized Implementation:** Using **Path Compression** and **Union by Rank/Size**, the time complexity per operation becomes nearly constant, specifically **$O(\alpha(N))$**, where α is the inverse Ackermann function (which grows extremely slowly and is effectively < 5 for all practical values of N).

3. Optimization Techniques

- **Union by Rank/Height:** To keep the tree shallow, always attach the shorter tree under the root of the taller tree. This ensures the height of any tree is at most $O(\log N)$.
- **Path Compression:** During a Find operation, make every node on the path point directly to the root. This flattens the structure, significantly speeding up future operations.

4. Pseudo-code for Operations

C++

```
// Initialization
void makeSet(int n) {
    for (int i = 1; i <= n; i++) {
        parent[i] = i;
        rank[i] = 0; // or size[i] = 1
    }
}

// Find with Path Compression
int find(int i) {
    if (parent[i] == i)
        return i;
    return parent[i] = find(parent[i]); // Path compression
}

// Union by Rank
```

```
void unite(int i, int j) {
    int root_i = find(i);
    int root_j = find(j);
    if (root_i != root_j) {
        if (rank[root_i] < rank[root_j])
            parent[root_i] = root_j;
        else if (rank[root_i] > rank[root_j])
            parent[root_j] = root_i;
        else {
            parent[root_i] = root_j;
            rank[root_j]++;
        }
    }
}
```

5. Simulation / Application Logic

- **Scenario Application:** If a question asks to "check if two computers are in the same network" or "automatically group items as they arrive," DSU is the most efficient choice.
- **Simulation Steps:** 1. Start with each element as its own root.
  2. For a `Union(x, y)` command, find the roots of x and y.
  3. Check ranks/heights.
  4. Attach the root of the smaller tree to the root of the larger tree.
  5. If a `Find(x)` occurs, trace up to the root and perform path compression if specified.

Summary Table for Exam Prep

Topic	Complexity	Best For	Key Logic
DSU	$O(\alpha(N))$	Connectivity, Cycles	Path Compression + Union by Rank
Binary Heap	$O(\log N)$	Priority Queues	Percolate Up/Down

Hashing	$O(1)$ (Avg)	Fast Key Lookups	Hash Functions & Collisions
AVL Tree	$O(\log N)$	Sorted Data Searches	Rotations (LL, RR, LR, RL)
Sparse Table	$O(1)$ Query	Static Range Min/Max	Power of 2 intervals

String hashing:

Rabin Karp String Matching - Rolling Hash

Source 0: https://cp-algorithms.com/string/string-hashing.html
Source 1: https://cp-algorithms.com/string/rabin-karp.html
Source 2:
https://www.geeksforgeeks.org/dsa/introduction-to-rolling-hash-data-structures-and-algorithms/

Trie Data Structure

Source 1: https://www.geeksforgeeks.org/dsa/trie-insert-and-search/
Source 2: https://www.geeksforgeeks.org/dsa/trie-delete/

Problem 1: Finding the number of Strings that have the given common prefix.

Alternate Source: https://www.tutorialspoint.com/data_structures_algorithms/tries.htm